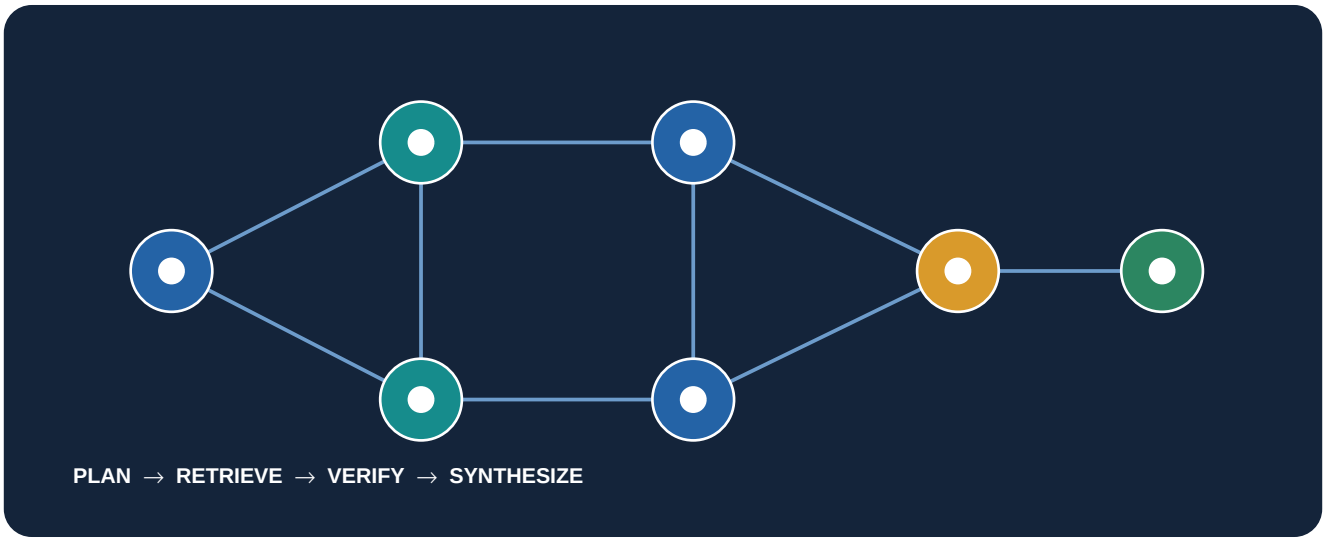
ATLAS

Multi-Agent Research Intelligence System

Complete architecture, phase-wise implementation roadmap, engineering standards and production readiness plan



Project objective

Build a research assistant that decomposes complex questions, coordinates specialized agents, searches internal and external knowledge, verifies claims against source evidence, identifies uncertainty and produces traceable research reports with citations.

Blueprint area	Included in this document
Architecture	Agent topology, shared state, retrieval paths, infrastructure and data flow
Implementation	13 development phases with deliverables, tasks, tests and exit criteria
Engineering	Modular Python structure, configuration, persistence, observability and security
Quality	Evidence provenance, evaluation metrics, failure handling and deployment checklist

Prepared as a step-by-step engineering blueprint. The implementation is intentionally phased: build and validate the foundations before introducing agent complexity.

Contents

A structured guide to the system architecture, retrieval design, implementation sequence and production operations.

SECTION	WHAT YOU'LL FIND
01	Executive overview and target use case
02	System architecture and agent responsibilities
03	Research workflow and state transitions
04	Retrieval and knowledge architecture
05	Agent contracts and orchestration
06	Persistence and data model
07	Production reliability, security and governance
08	Complete phase-wise implementation roadmap
09	Initial implementation patterns
10	API design and user experience
11	Evaluation strategy and acceptance metrics
12	Deployment topology and operational model
13	Key design decisions and production readiness
14	Recommended first vertical slice and references

Implementation phases

PHASE	DELIVERABLE
01	Project bootstrap, configuration and health checks
02	Domain models, PostgreSQL and repositories
03	Document ingestion and extraction
04	Embeddings and Qdrant vector storage
05	Hybrid retrieval and CrossEncoder reranking
06	LLM provider abstraction and structured output
07	Specialized research agents
08	LangGraph supervisor and orchestration
09	Evidence verification and critic
10	Synthesis, citations and report generation

1. Executive overview

ATLAS is a production-oriented Multi-Agent Retrieval-Augmented Generation (RAG) system for research. It combines agent orchestration with two knowledge acquisition paths: internal document retrieval and external research. Agents specialize in planning, source discovery, domain analysis, verification, critique and synthesis.

The system is designed for complex, multi-part questions where one retrieval call and one LLM response are not sufficient. The supervisor coordinates the workflow, while every agent works within a defined responsibility and returns structured findings with source provenance.

What ATLAS will do

- Interpret a research question and identify scope, constraints, time horizon and expected output.
- Break the question into research tasks and assign work to suitable agents.
- Search uploaded documents and external sources using fit-for-purpose retrieval tools.
- Use hybrid retrieval (dense semantic search plus sparse keyword search) and reranking where appropriate.
- Normalize findings into evidence records with source, location, date and retrieval metadata.
- Verify whether sources actually support claims and flag contradictions or weak evidence.
- Run a critic pass to identify gaps, unsupported conclusions and missing perspectives.
- Produce a structured report with citations, limitations and a traceable evidence trail.
- Persist research runs, tasks, agent executions and artifacts so work can be inspected and resumed.

Non-goals for the first release

- The system will not autonomously publish research or take consequential external actions.
- It will not treat LLM-generated confidence as a calibrated probability.
- It will not assume that more agents automatically mean better research.
- It will not treat retrieved web pages or uploaded files as instructions for the agents.
- It will not claim exhaustive coverage of the web or of any literature corpus.

Design principle: use the smallest number of agents that reliably completes the task. Parallelize independent research tasks, but keep shared state, evidence contracts and quality gates explicit.

2. Target use case and example

Example user request

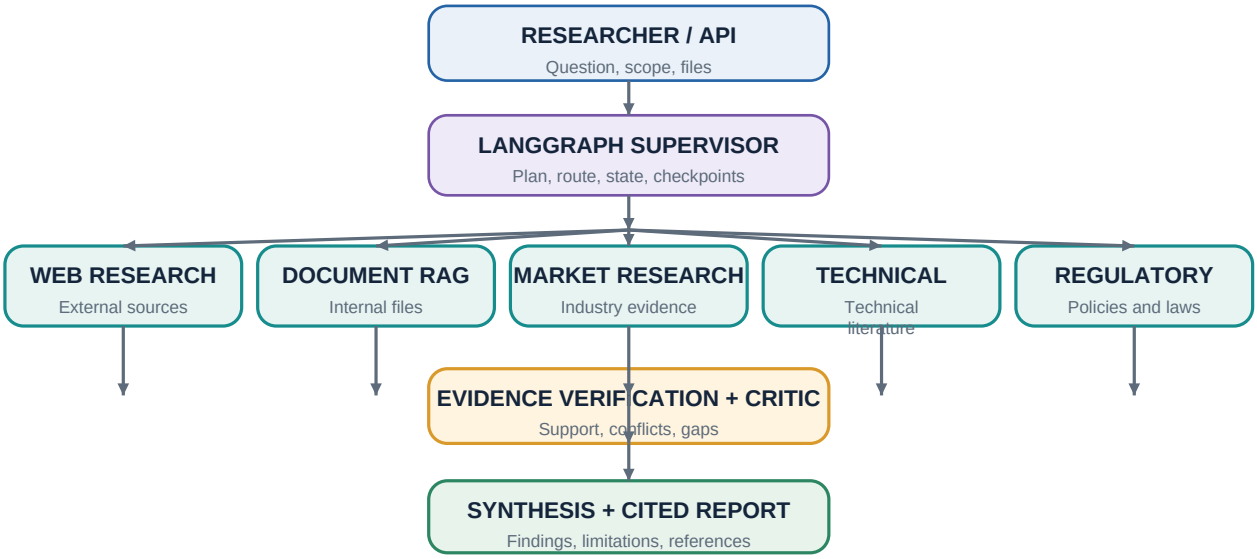
“Research the adoption of Generative AI in healthcare. Analyze applications, market trends, technical developments, regulatory considerations, risks and future opportunities. Provide evidence-backed findings with citations and clearly identify uncertainty.”

Expected report sections

Section	Expected content
Executive summary	Short synthesis of the most important findings and their implications.
Scope and method	Question interpretation, date range, source types and search limitations.
Applications	Clinical documentation, patient engagement, medical imaging support, research and operations, where supported by sources.
Market and adoption	Adoption signals, investment or market estimates, clearly dated and attributed.
Technical landscape	Models, architectures, evaluation approaches and deployment patterns.
Regulation and governance	Applicable rules, guidance, jurisdiction and effective dates.
Risks and limitations	Safety, privacy, bias, reliability, workflow and implementation risks.
Opportunities	Evidence-supported opportunities and conditions for realizing them.
References and evidence appendix	Source list, claim-to-source mapping, publication dates and caveats.

3. System architecture

The architecture has five main layers: API entry, orchestration, specialized agents, retrieval and infrastructure. The supervisor controls the workflow but does not need to perform every research task itself.



3.1 Main components and responsibilities

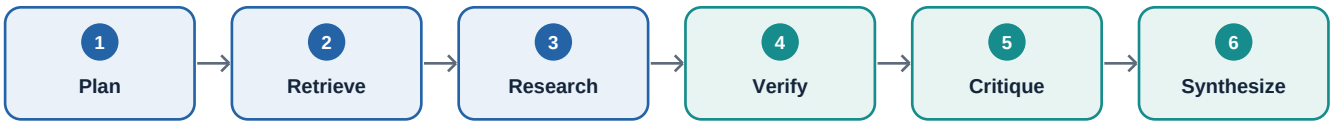
Component	Responsibility	Key output
FastAPI gateway	Accept requests, validate inputs, authenticate users and expose job/report endpoints.	Validated research request
Research application service	Create research runs, persist initial records and schedule execution.	Research job ID
LangGraph supervisor	Coordinate planning, routing, parallel work, retries and checkpoints.	Workflow state and decisions
Planning agent	Decompose the question into bounded, non-duplicative tasks.	Research plan
Research agents	Investigate assigned questions using authorized tools and sources.	Structured findings and evidence
Verifier	Check claim support, source quality, dates and conflicts.	Verified/rejected claims
Critic	Identify coverage gaps, reasoning issues and unsupported conclusions.	Critique and gap list
Synthesizer	Generate a readable report using verified claims and source references.	Cited report
Persistence layer	Store jobs, tasks, executions, evidence, sources and artifacts.	Durable history

3.2 Recommended agent topology

Agent	Core responsibility	Should it be separate?
Supervisor	Routing, lifecycle and workflow control.	Yes. It owns orchestration.
Planner	Task decomposition and scope definition.	Yes, especially for complex questions.
Web researcher	External web search and source extraction.	Yes, because its tools and trust boundary differ.
Document researcher	Search private/internal documents.	Yes, because access control and retrieval differ.
Domain researchers	Market, technical and regulatory analysis.	Initially configurable roles; split when evaluation proves value.
Evidence verifier	Claim-to-source support and provenance checks.	Yes. Keep verification independent from synthesis.
Critic	Coverage, contradictions and research gaps.	Can share a model provider but should be a distinct workflow step.
Synthesizer	Compose the final report from validated evidence.	Yes. It should not invent new evidence.

Agent count is a design choice, not a quality metric. Begin with planner, web researcher, document researcher, verifier, critic and synthesizer. Add market, technical and regulatory specialists as task types and evaluation results justify them.

4. Research workflow and state transitions



4.1 End-to-end workflow

Step	Action	Decision / output
1. Intake	Validate question, scope, dates, requested depth and available files.	Research request accepted or rejected with validation errors.
2. Plan	Planner creates research dimensions, subtasks, source requirements and success criteria.	Structured plan with task IDs and dependencies.
3. Route	Supervisor assigns tasks to internal-document or external-research agents.	Agent assignments and tool permissions.
4. Retrieve	Agents search, fetch and extract relevant material.	Candidate source records and passages.
5. Analyze	Agents produce concise findings linked to evidence IDs.	Structured findings, not free-floating prose.
6. Verify	Verifier checks each material claim against its cited evidence.	Supported, partially supported, contradicted or unsupported.
7. Critique	Critic checks coverage, diversity, contradictions and unanswered tasks.	Gap list and revision recommendations.
8. Revise	Supervisor selectively reopens tasks if budget and retry limits allow.	Additional evidence or explicit unresolved gaps.
9. Synthesize	Synthesizer writes report from validated findings.	Report with inline citations and reference list.
10. Persist	Save report, sources, task outcomes and execution trace.	Retrievable research record.

4.2 Conditional routing rules

- If the request is simple and narrow, use a compact retrieval-and-answer path rather than launching multiple agents.
- If internal files are attached or selected, route relevant tasks to the document researcher.
- If current or time-sensitive facts are requested, use external research and record publication and retrieval dates.
- If evidence is insufficient, allow a bounded retrieval retry with a rewritten query or a different source type.
- If claims conflict, preserve both positions and report the disagreement rather than forcing a false consensus.
- If retry budget is exhausted, synthesize with an explicit limitations section or mark the run incomplete.

5. Retrieval and knowledge architecture

Research RAG needs two distinct ingestion and retrieval paths. They converge into a common evidence model so downstream agents can reason over consistent records without losing provenance.

5.1 Internal document RAG

Stage	Implementation responsibility
Upload	Validate file type, size, tenant ownership and malware scanning policy.
Extraction	Extract text, page numbers, tables and document metadata; preserve page-level locations.
Normalization	Clean extraction artifacts while retaining original text and source coordinates.
Chunking	Split by document structure first, then enforce token/character limits with overlap.
Embedding	Generate vectors using a configured embedding model and version.
Indexing	Store vectors and metadata in Qdrant; maintain sparse index for keyword retrieval.
Retrieval	Apply tenant and document ACL filters before returning candidate chunks.
Reranking	Rerank candidates using a CrossEncoder or another evaluated reranker.
Evidence capture	Store exact passage, page/section, document ID and content hash.

5.2 External research path

Stage	Implementation responsibility
Query generation	Create focused search queries from a research task.
Search	Call a search provider through an adapter with rate limits and timeouts.
Fetch	Fetch allowed pages or source records; respect provider terms and robots/access rules.
Extraction	Extract title, author/publisher, date, canonical URL and relevant passages.
Quality checks	Deduplicate, classify source type and flag missing or ambiguous dates.
Evidence capture	Store source snapshot metadata and passage references; preserve URL and retrieval time.

5.3 Hybrid retrieval

Use dense retrieval for semantic similarity and sparse retrieval for exact terms, acronyms, names, model numbers and phrases. Combine candidates, deduplicate them, then rerank. The exact fusion strategy and top-k values must be selected through evaluation, not guessed.

Retrieval method	Strength	Typical weakness
Dense vector search	Finds semantically related passages even when wording differs.	Can miss exact identifiers, rare terms or precise phrases.
Sparse / BM25	Strong for exact words, names, acronyms and technical terms.	May miss semantically equivalent wording.
Hybrid fusion	Combines complementary signals.	Needs tuning and careful deduplication.

Retrieval method	Strength	Typical weakness
CrossEncoder reranking	Scores query-passage pairs with richer interaction.	More expensive than first-stage retrieval.

5.4 Common evidence record

Every agent should return evidence references rather than only prose. A shared evidence record might contain:

```
{ "evidence_id": "ev_...", "research_id": "res_...", "task_id": "task_...", "source_id": "src_...",
"source_type": "web | internal_document | paper", "claim_text": "A concise factual claim", "passage": "Exact
supporting passage or faithful excerpt", "locator": {"page": 12, "section": "Methods"}, "source_url":
"https://...", "title": "Source title", "publisher": "Publisher or organization", "published_at":
"2026-01-15", "retrieved_at": "2026-11-02T...", "content_hash": "sha256:...", "agent_id":
"technical_researcher", "verification_status": "pending" }
```

Keep source metadata separate from the model's interpretation. A source passage is evidence; an agent's summary is an interpretation. Store both with links between them.

5.5 Chunking and metadata recommendations

- Prefer structure-aware chunking: headings, paragraphs, tables, abstracts and sections.
- Store document ID, tenant ID, page number, section heading, source type, publication date and embedding version.
- Use stable chunk IDs and content hashes to support deduplication and re-indexing.
- Avoid excessively small chunks that lose context and excessively large chunks that dilute retrieval.
- Keep original documents in object storage and extracted text/metadata in durable storage.
- Apply access-control filters before retrieval, not after the model has seen the content.

6. Agent contracts and orchestration

LangGraph should coordinate agents through typed state and explicit node outputs. The state is a workflow contract, not a dumping ground for entire documents or long chat transcripts.

6.1 Shared state design

```
class ResearchState(TypedDict): research_id: str user_query: str scope: dict plan: dict tasks: list[dict]
task_results: Annotated[list[dict], operator.add] evidence_records: Annotated[list[dict], operator.add]
verified_claims: list[dict] rejected_claims: list[dict] gaps: list[dict] critique: dict report: dict | None
retry_count: int status: str errors: list[dict]
```

Use reducers for fields that receive parallel updates. Do not let multiple nodes overwrite the same list without a defined merge strategy. Store large payloads in PostgreSQL or object storage and pass IDs or compact summaries through graph state.

6.2 Agent input/output contract

Contract field	Purpose
agent_id	Stable identity of the agent implementation.
task_id	Research task being executed.
objective	Specific question the agent must answer.
allowed_tools	Explicit list of tools the agent may use.
source_scope	Allowed internal collections, tenant scope or external domains.
output_schema	Pydantic schema defining required output fields.
evidence_ids	References to evidence records supporting findings.
status	Succeeded, partial, failed or skipped.
usage	Token, latency and tool-call accounting.

6.3 Supervisor responsibilities

- Create the plan and validate that tasks are sufficiently specific.
- Choose which agents are necessary and which tasks can run in parallel.
- Enforce maximum parallel tasks, timeouts, retry budgets and token/cost budgets.
- Track task status and checkpoint state for recovery.
- Route weak or conflicting evidence to targeted follow-up research.
- Prevent synthesis until required verification and quality checks have completed.
- Persist execution events and produce a final run status.

6.4 Parallelism and dependencies

Independent research tasks can run in parallel. Tasks that depend on earlier findings should be sequenced. For example, the planner must complete before task fan-out; evidence verification must follow evidence collection; synthesis must follow verification and critique.

7. Persistence and data model

Use PostgreSQL as the system of record. Qdrant stores vector representations and searchable metadata. Object storage holds original documents and generated report files. Redis supports caching, rate limiting and queue coordination where needed.

7.1 Core entities

Entity	Important fields	Relationship
Research	id, owner_id, query, scope, status, created_at, completed_at	One research run has many tasks, sources and reports.
ResearchTask	id, research_id, objective, agent_type, status, priority, attempts	Belongs to a research run.
AgentExecution	id, task_id, agent_id, model, started_at, ended_at, tokens, error	One task can have multiple attempts.
Document	id, owner_id, storage_key, filename, mime_type, checksum, status	Can produce many chunks and evidence records.
Source	id, research_id, source_type, url, title, publisher, published_at, retrieved_at	Can support many evidence records.
Evidence	id, source_id, task_id, passage, locator, claim, verification_status	Connects findings to source passages.
Report	id, research_id, version, storage_key, format, created_at	A research run can have report versions.
Checkpoint	thread_id, checkpoint_id, state_ref, created_at	Supports workflow recovery.

7.2 Suggested database relationships

```
Research 1 ■■■ N ResearchTask ResearchTask 1 ■■■ N AgentExecution Research 1 ■■■ N Source Source 1 ■■■ N Evidence ResearchTask 1 ■■■ N Evidence Research 1 ■■■ N Report Research 1 ■■■ N Checkpoint
```

7.3 Repository pattern

Application services should depend on repository interfaces such as ResearchRepository, TaskRepository, EvidenceRepository and ReportRepository. SQLAlchemy implementations live in infrastructure. This makes unit tests easier and avoids coupling agents to database sessions.

7.4 Persistence rules

- Persist the research run before scheduling work so every job has a durable ID.
- Use database transactions for related state changes.
- Use idempotency keys for request submission and task scheduling.
- Store timestamps in UTC and convert for display.
- Use migrations for schema changes; do not create production schema implicitly at startup.
- Record model/provider versions and prompt versions for reproducibility.
- Apply tenant ownership checks to every read and write path.

8. Production reliability, security and governance

Research systems process untrusted content and can invoke external tools. Treat source text as data, enforce permissions outside the LLM, and make execution bounded and observable.

8.1 Security controls

Risk	Control
Prompt injection in web pages or PDFs	Treat retrieved content as untrusted data; never let it override system instructions or tool permissions.
Cross-tenant data leakage	Tenant-scoped retrieval filters, repository authorization and tests for isolation.
Secret exposure	Environment/secret manager, SecretStr, redaction and no secrets in logs.
Unsafe file uploads	File type/size validation, malware scanning policy, isolated parsing and storage controls.
Unrestricted web access / SSRF	Allowlisted schemes/domains where applicable, block private IP ranges and validate redirects.
Excessive tool use or cost	Per-run budgets, rate limits, timeouts, max iterations and concurrency limits.
Sensitive research content	Retention policy, access controls, encryption in transit and at rest.
Unsafe report rendering	Escape untrusted text and sanitize generated HTML/Markdown where rendered.

8.2 Reliability patterns

- Timeouts on every network and model call.
- Bounded retries with exponential backoff and jitter for transient failures.
- Circuit breakers or provider fallback for repeated upstream failures.
- Idempotent ingestion and task execution.
- Checkpointing and resumable workflow execution.
- Dead-letter handling for jobs that repeatedly fail.
- Graceful partial completion when a non-critical source or agent fails.
- Explicit cancellation and deadline propagation.

8.3 Source quality and research governance

- Classify sources by type and provenance; do not treat all sources as equally authoritative.
- Record publication date and retrieval date separately.
- Prefer primary sources for regulations, official statistics and original research when available.
- Flag paywalled, inaccessible, undated or secondary sources.
- Preserve contradictory evidence and describe why sources differ.
- Avoid presenting an agent's confidence score as a calibrated statistical confidence.
- Make research limitations visible in the final report.

8.4 Observability

Signal	Examples
Workflow	Research run duration, task completion rate, retries, incomplete runs.
Agent	Agent latency, model, prompt version, tool calls, token usage, failure reason.
Retrieval	Recall@k, reranker score distribution, empty retrieval rate, source diversity.
Evidence	Supported claim rate, unsupported claim rate, citation coverage, stale-source rate.
Infrastructure	API latency, queue depth, database pool usage, Qdrant health, Redis health.
Cost	Cost per research run, per agent and per report section.

9. Complete phase-wise implementation roadmap

The roadmap is ordered to reduce rework. Each phase has a clear deliverable and exit criteria. The project should not advance just because code exists; it should advance when the phase is testable and its key assumptions are validated.

Phase	Focus	Main deliverable
1	Project bootstrap	Runnable FastAPI application, settings and logging
2	Domain models + database	Persisted research, task, source and evidence records
3	Document ingestion	Reliable PDF/DOCX/TXT ingestion pipeline
4	Embeddings + Qdrant	Versioned vector indexing and metadata filters
5	Hybrid retrieval + reranking	Evaluated dense + sparse retrieval pipeline
6	LLM abstraction	Configurable model providers and structured outputs
7	Specialized agents	Planner, web/document/domain researchers
8	LangGraph orchestration	Supervisor, parallel execution, routing and checkpoints
9	Verification + critic	Evidence validation, contradiction and gap detection
10	Synthesis + citations	Traceable research reports and source appendix
11	API + background workers	Asynchronous jobs, status, cancellation and artifact retrieval
12	Testing + evaluation	Automated quality, regression and reliability evaluation
13	Deployment + monitoring	Secure deployable service with operational visibility

Suggested delivery approach

Build a narrow vertical slice early: one question, one internal document collection, one web source adapter, one researcher, one verifier and one cited report. Then expand source coverage and agent specialization based on measured gaps. This avoids building a large orchestration framework before proving retrieval and evidence quality.

Phase 1. Project bootstrap and configuration

Goal: Create a runnable Python application with consistent configuration, logging and health checks.

Implementation tasks

- Create the repository, virtual environment and package structure.
- Define pyproject.toml, development dependencies, formatting and test settings.
- Implement Pydantic Settings with environment-based configuration and secret handling.
- Add structured JSON logging with request and research correlation IDs.
- Create FastAPI app factory, lifespan handling and liveness endpoint.
- Add .env.example and .gitignore; keep real secrets out of source control.

Deliverables

- Runnable FastAPI service and Swagger UI.
- Central settings module and structured logger.
- Initial unit test and developer setup instructions.

Tests and validation

- Settings load defaults and environment overrides correctly.
- Health endpoint returns expected response.
- Application starts and shuts down cleanly.
- Secret values are not emitted in logs.

Exit criteria

- Fresh environment can install the project and start the API.
- Configuration is not duplicated across modules.
- A basic CI check can run tests and linting.

Phase 2. Domain models, database and repositories

Goal: Define the core research entities and durable persistence before agent workflows are introduced.

Implementation tasks

- Define Research, ResearchTask, AgentExecution, Document, Source, Evidence and Report schemas.
- Implement SQLAlchemy 2.x async database engine and session management.
- Create Alembic migrations and a migration workflow.
- Define repository interfaces in the application/domain boundary.
- Implement PostgreSQL repositories and transaction boundaries.
- Add idempotency keys, ownership fields, UTC timestamps and status enums.

Deliverables

- Database schema and migrations.
- Repository implementations and dependency injection.
- CRUD use cases for research runs and task records.

Tests and validation

- Repository unit/integration tests against a disposable PostgreSQL database.
- Migration upgrade and downgrade test.
- Tenant ownership and duplicate idempotency tests.
- Transaction rollback test.

Exit criteria

- Research run and task lifecycle persist correctly.
- No agent module opens its own database connection directly.
- Migrations reproduce the schema from an empty database.

Phase 3. Document ingestion pipeline

Goal: Convert uploaded research material into normalized, traceable text and metadata.

Implementation tasks

- Define upload validation for size, MIME type, extension and ownership.
- Implement loaders for PDF, DOCX and TXT with page/section locators.
- Add extraction normalization while preserving original text and source coordinates.
- Implement structure-aware chunking with configurable size and overlap.
- Generate stable document and chunk IDs plus content hashes.
- Persist document status and extraction metadata; store originals in object storage.
- Handle scanned PDFs as a separate OCR capability, not as silently successful text extraction.

Deliverables

- Loader interfaces and format-specific implementations.
- Ingestion pipeline with status tracking.
- Document/chunk metadata schema and object storage adapter.

Tests and validation

- Test normal, empty, corrupted, scanned and oversized files.
- Verify page/section locators survive chunking.
- Test duplicate uploads and content-hash behavior.
- Test cleanup of temporary files after failures.

Exit criteria

- Supported file types ingest deterministically.
- Every chunk maps to a document and source locator.
- Failed extraction is visible and does not create a falsely complete document.

Phase 4. Embeddings and Qdrant vector storage

Goal: Create a versioned embedding and indexing layer for internal document retrieval.

Implementation tasks

- Implement an embedding provider interface and Sentence Transformers adapter.
- Record embedding model name, revision and vector dimension.
- Create Qdrant collection initialization and schema management scripts.
- Upsert chunks with stable IDs and tenant/document metadata.
- Implement metadata filters and deletion/re-indexing workflows.
- Make ingestion idempotent and track indexing status.

Deliverables

- Embedding service and Qdrant adapter.
- Collection bootstrap script.
- Indexing and re-indexing use cases.

Tests and validation

- Vector dimension compatibility tests.
- Upsert duplicate and update tests.
- Tenant/document filter tests.
- Delete and re-index tests.

Exit criteria

- Documents can be indexed and retrieved by vector similarity.
- Indexing is repeatable without duplicate logical chunks.
- Metadata filtering is enforced in the retrieval adapter.

Phase 5. Hybrid retrieval and reranking

Goal: Improve search relevance by combining semantic retrieval, keyword retrieval and reranking.

Implementation tasks

- Implement dense retriever over Qdrant.
- Implement sparse BM25 retrieval with a defined index lifecycle.
- Fuse results using a measured strategy such as reciprocal rank fusion.
- Deduplicate candidates by stable chunk ID.

- Implement CrossEncoder reranking behind a provider interface.
- Add query filters, top-k configuration and source diversity controls.
- Create a small gold evaluation set of queries and relevant passages.

Deliverables

- Dense, sparse and hybrid retriever interfaces.
- Reranker adapter.
- Retrieval evaluation dataset and baseline report.

Tests and validation

- Measure Recall@k, MRR or nDCG on the gold set.
- Test exact identifiers and semantic paraphrases.
- Test no-result and metadata-filter scenarios.
- Measure retrieval latency and memory use.

Exit criteria

- Hybrid retrieval demonstrably improves or complements the baseline.
- Top-k and rerank settings are config-driven.
- Retrieval returns provenance metadata with each passage.

Phase 6. LLM provider abstraction and structured output

Goal: Allow agents to use supported LLM providers through a common, testable interface.

Implementation tasks

- Create an LLM factory driven by configuration.
- Add provider adapters for the selected initial provider and one fallback provider.
- Define typed Pydantic schemas for planner, researcher, verifier, critic and synthesis outputs.
- Add timeout, retry, token and cost accounting wrappers.
- Version prompts and store prompt identifiers with executions.
- Validate model outputs and handle malformed JSON or schema failures.

Deliverables

- LLM interface and provider factory.
- Typed output schemas and prompt templates.
- Usage and error instrumentation.

Tests and validation

- Mock provider tests for success, timeout and malformed outputs.
- Schema validation tests.

- Provider selection tests.
- Secret redaction tests.

Exit criteria

- Agents do not instantiate provider SDKs directly.
- Provider/model can be changed through settings.
- Structured output failures are explicit and recoverable.

Phase 7. Specialized research agents

Goal: Implement focused agents that return structured findings and evidence references.

Implementation tasks

- Implement planner agent to create task plans with scope and success criteria.
- Implement web researcher with search/fetch adapters and source metadata extraction.
- Implement document researcher using the hybrid retrieval pipeline.
- Implement domain researcher templates for market, technical and regulatory topics.
- Define common AgentInput and AgentResult contracts.
- Restrict each agent to its allowed tools and source scope.
- Add deterministic mock tools for unit tests.

Deliverables

- Planner and researcher implementations.
- Agent contracts and schemas.
- Tool permission map and prompt versions.

Tests and validation

- Test each agent independently with fixed fixtures.
- Verify every factual finding references evidence IDs.
- Test empty retrieval, provider error and partial results.
- Test prompt-injection content is treated as untrusted data.

Exit criteria

- Each agent has one bounded responsibility.
- Outputs conform to schemas and contain provenance.
- Agent code is testable without live web or LLM calls.

Phase 8. LangGraph supervisor and orchestration

Goal: Coordinate planning, task fan-out, parallel research, state updates, retries and checkpoints.

Implementation tasks

- Define typed ResearchState with reducers for parallel result accumulation.
- Implement graph nodes for intake, planning, task routing and agent execution.
- Use bounded parallel execution for independent tasks.
- Implement conditional edges for retries, gaps, failures and completion.
- Add checkpoint persistence and research thread identifiers.
- Enforce max agents, max tasks, deadlines, retries and budget limits.
- Persist task and agent execution events outside transient graph state.

Deliverables

- Compiled LangGraph workflow.
- Supervisor routing and bounded parallel execution.
- Checkpoint and recovery integration.

Tests and validation

- Graph integration tests for success and partial failure.
- Test parallel updates do not overwrite each other.
- Interrupt/restart and checkpoint recovery tests.
- Test max retry and deadline enforcement.

Exit criteria

- A research run can complete and resume from a checkpoint.
- Parallel tasks produce deterministic merged state.
- Workflow cannot enter synthesis before required prerequisites.

Phase 9. Evidence verification and critic

Goal: Separate evidence checking from research generation and identify gaps before report synthesis.

Implementation tasks

- Implement claim extraction or claim normalization from agent findings.
- Verify each material claim against its linked source passage.
- Classify support as supported, partially supported, contradicted or unsupported.
- Evaluate source provenance, date relevance and source-type suitability.
- Detect duplicated claims and contradictory evidence.
- Implement critic checks for unanswered tasks, weak coverage and one-sided evidence.
- Route only actionable gaps back to targeted research, within a retry budget.

Deliverables

- Verifier service and structured verification records.
- Critic service and gap schema.

- Supervisor revision loop.

Tests and validation

- Build labeled claim-evidence test cases.
- Measure claim support precision/recall on reviewed examples.
- Test contradictory source cases and stale sources.
- Test that unsupported claims are excluded or explicitly qualified.

Exit criteria

- Every material report claim is linked to evidence or marked as analysis/opinion.
- Contradictions and gaps remain visible.
- Critic loop is bounded and does not run indefinitely.

Phase 10. Synthesis, citations and report generation

Goal: Produce coherent research reports from verified claims while preserving traceability.

Implementation tasks

- Define report schema and section templates.
- Synthesize only from verified claims and clearly labeled analysis.
- Generate inline citations linked to source IDs and exact locators.
- Generate reference list with publisher, title, URL and publication/retrieval dates.
- Include scope, methodology, limitations, conflicts and unresolved gaps.
- Render Markdown first; add PDF export as a separate reporting adapter.
- Persist report versions and the evidence snapshot used for each version.

Deliverables

- Synthesis agent and report schema.
- Markdown report renderer and citation resolver.
- Versioned report artifacts.

Tests and validation

- Citation resolution test: every citation points to a stored source/evidence record.
- Test no invented references or broken links.
- Review report readability and claim coverage.
- Test conflicting evidence is represented fairly.

Exit criteria

- Reports are reproducible from stored evidence and prompt versions.
- Every citation resolves to a source record.

- Limitations and unresolved questions are visible.

Phase 11. FastAPI endpoints and background workers

Goal: Expose research as an asynchronous service suitable for longer-running workflows.

Implementation tasks

- Add POST endpoint to create a research run and return a job ID.
- Add GET endpoints for research status, task progress and report retrieval.
- Add document upload and ingestion status endpoints.
- Use a durable queue/worker model for long-running research jobs.
- Implement cancellation, deadlines, retry policy and dead-letter handling.
- Add authentication, authorization, tenant scoping and request rate limits.
- Stream progress through polling or server-sent events where useful.

Deliverables

- Versioned API routes and request/response schemas.
- Worker process and queue integration.
- Status, cancellation and report retrieval flows.

Tests and validation

- API contract tests and auth tests.
- Worker restart and duplicate-delivery tests.
- Cancellation and timeout tests.
- Load test concurrent research requests.

Exit criteria

- API requests return quickly while research runs asynchronously.
- Jobs are durable and observable across worker restarts.
- Unauthorized users cannot access another tenant's data.

Phase 12. Testing, evaluation and quality gates

Goal: Measure retrieval, agent, evidence and end-to-end report quality rather than relying on anecdotal examples.

Implementation tasks

- Create unit tests for domain logic, repositories, adapters and agents.
- Create integration tests for PostgreSQL, Qdrant, Redis and object storage.
- Create end-to-end research scenarios with fixed source fixtures.
- Build retrieval gold sets and human-reviewed claim/evidence datasets.

- Evaluate retrieval Recall@k, MRR/nDCG and reranker lift.
- Evaluate citation coverage, claim support, contradiction handling and answer completeness.
- Track latency, token usage, cost, retry rate and failure rate.
- Add regression tests for prompt/model changes.

Deliverables

- Test suites and CI pipeline.
- Evaluation datasets and scoring scripts.
- Quality dashboard or evaluation report.

Tests and validation

- Run tests in CI with deterministic mocks.
- Run periodic model-based evaluations on fixed datasets.
- Human-review a sample of research reports.
- Set release thresholds from measured baseline performance.

Exit criteria

- Critical security and correctness tests pass.
- Quality metrics are tracked over time.
- Model or prompt changes cannot silently regress key behavior.

Phase 13. Deployment, monitoring and operations

Goal: Deploy a secure, observable service with controlled scaling, recovery and maintenance.

Implementation tasks

- Containerize API and workers separately.
- Deploy PostgreSQL, Qdrant and Redis with backups and health checks.
- Configure object storage lifecycle and encryption.
- Use managed secrets and separate development/staging/production settings.
- Add OpenTelemetry traces and metrics across API, graph nodes and tools.
- Create dashboards and alerts for latency, failures, queue depth and cost.
- Define backup/restore, retention, incident and rollback procedures.
- Document capacity limits, model-provider dependencies and operational runbooks.

Deliverables

- Docker images and deployment manifests.
- Monitoring dashboards and alert rules.
- Backup/restore and incident runbooks.
- Production readiness review.

Tests and validation

- Container health and dependency readiness tests.
- Backup restoration drill.
- Load and failure-injection tests.
- Security review and dependency scanning.

Exit criteria

- Service can be deployed reproducibly.
- Critical alerts and recovery procedures are tested.
- Operational ownership and limits are documented.

10. Initial implementation patterns

The following snippets illustrate the intended contracts and module boundaries. They are architectural starting points, not a complete drop-in implementation of every phase. Dependency versions and adapter APIs should be pinned and tested when each phase is implemented.

10.1 Typed agent result

```
from datetime import datetime from typing import Literal from pydantic import BaseModel, Field, HttpUrl class EvidenceReference(BaseModel): evidence_id: str source_id: str locator: dict = Field(default_factory=dict) class Finding(BaseModel): claim: str explanation: str evidence: list[EvidenceReference] = Field(default_factory=list) caveats: list[str] = Field(default_factory=list) class AgentResult(BaseModel): agent_id: str task_id: str status: Literal["succeeded", "partial", "failed"] findings: list[Finding] = Field(default_factory=list) unresolved_questions: list[str] = Field(default_factory=list) started_at: datetime completed_at: datetime | None = None error_code: str | None = None
```

10.2 Application-level ports

```
from typing import Protocol class ResearchRepository(Protocol): async def create(self, research): ... async def get(self, research_id: str): ... async def update_status(self, research_id: str, status: str): ... class EvidenceRepository(Protocol): async def add_many(self, evidence_records: list): ... async def get_by_ids(self, evidence_ids: list[str]): ... class Retriever(Protocol): async def search(self, query: str, filters: dict, top_k: int): ... class ResearchTool(Protocol): async def search(self, query: str, limit: int): ...
```

10.3 Graph-level conceptual flow

```
START -> validate_request -> plan_research -> dispatch_research_tasks -> collect_agent_results -> verify_evidence -> critique_coverage |-- gaps and retry budget available -> targeted_research |-- otherwise -> synthesize_report -> persist_report -> END
```

10.4 Configuration example

Setting	Example	Why it exists
LLM_PROVIDER	groq	Select provider adapter.
LLM_MODEL	provider-specific model ID	Control model without changing agent code.
EMBEDDING_MODEL	all-MiniLM-L6-v2	Version the embedding space.
QDRANT_COLLECTION	atlas_documents	Separate logical vector collection.
MAX_RESEARCH_TASKS	6	Bound parallel workload.
MAX_RESEARCH_RETRIES	2	Prevent unbounded loops.
MAX_RETRIEVAL_RESULTS	10	Control first-stage retrieval.
RERANK_TOP_K	5	Control context size after reranking.
REQUEST_TIMEOUT	60 seconds	Bound external calls.

Do not copy example values blindly into production. Establish them through workload tests, retrieval evaluation and provider cost measurements.

11. API design and user experience

Research can take longer than a normal HTTP request. The API should create a durable job and let the client inspect progress rather than holding one request open for the entire workflow.

Method + path	Purpose	Response concept
POST /api/v1/research	Create research run.	research_id, status, created_at
GET /api/v1/research/{id}	Get run summary and status.	status, scope, progress, timestamps
GET /api/v1/research/{id}/tasks	Inspect task and agent progress.	task list and execution states
GET /api/v1/research/{id}/report	Retrieve completed report.	report sections, citations, artifact link
POST /api/v1/documents	Upload internal source document.	document_id, ingestion_status
GET /api/v1/documents/{id}	Check ingestion status.	extraction and indexing state
POST /api/v1/research/{id}/cancel	Request cancellation.	cancellation_requested

Recommended research status lifecycle

```
queued → planning → researching → verifying → critiquing → synthesizing → completed
```

Terminal alternatives: failed, cancelled, partially_completed. Store a machine-readable reason and a user-readable explanation for terminal failures.

Recommended report UX

- Show a concise executive summary and a visible research scope.
- Provide progress by task, not just a generic spinner.
- Allow users to inspect sources and evidence behind each material claim.
- Show publication date and retrieval date for time-sensitive sources.
- Separate facts, interpretations and recommendations.
- Show conflicts, limitations and unanswered questions.
- Allow report export while preserving citation links.

12. Evaluation strategy and acceptance metrics

A production research system needs layered evaluation. A single answer-quality score hides whether a failure came from retrieval, evidence interpretation, orchestration or synthesis.

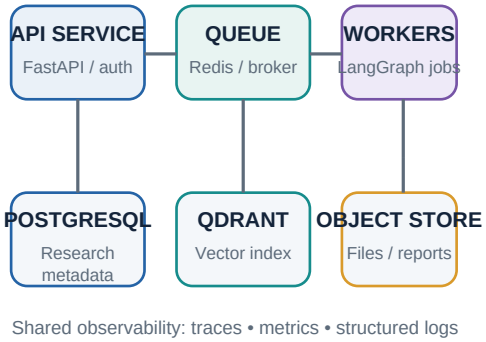
Layer	Metric / test	What it tells us
Ingestion	Extraction success, page/section locator accuracy	Whether source content is captured correctly.
Retrieval	Recall@k, MRR, nDCG	Whether relevant evidence is retrieved and ranked.
Agent	Schema validity, task completion, tool error rate	Whether agents execute their assigned tasks reliably.
Evidence	Claim support rate, unsupported claim rate	Whether findings are grounded in evidence.
Citation	Citation coverage, citation resolution rate	Whether report claims point to real sources.

Layer	Metric / test	What it tells us
Coverage	Research-question coverage, unresolved gap count	Whether requested dimensions were addressed.
Contradictions	Conflict detection precision/recall on labeled cases	Whether disagreements are preserved and identified.
Operations	Latency, cost/run, failure rate, retry rate	Whether the system is practical to operate.

Set thresholds after building a representative, human-reviewed evaluation set. Avoid inventing a universal target before seeing your corpus and use case.

13. Deployment topology and operational model

Keep the API and long-running workers separate so web request handling can scale independently from research execution.



13.1 Deployment components

Service	Scaling / operations
API	Scale on request throughput; keep handlers non-blocking.
Research workers	Scale on queue depth and provider rate limits; enforce per-run concurrency.
PostgreSQL	Managed or backed-up deployment, connection pooling, migrations and restore tests.
Qdrant	Persistent storage, collection backups/snapshots and index monitoring.
Redis	Use persistence/managed service appropriate to queue semantics; do not treat cache as source of truth.
Object storage	Versioning/lifecycle rules, encryption, access policy and artifact retention.
Observability	Correlated traces across API, graph nodes, retrievers, tools and model calls.

13.2 Environment separation

Environment	Purpose	Controls
Local	Developer feedback and unit tests.	Mocks, local containers, synthetic documents.
Development	Shared integration testing.	Non-production data, isolated credentials.
Staging	Production-like validation.	Realistic load, provider sandbox/limited budgets.
Production	User-facing research service.	Managed secrets, backups, alerts, access controls and change approvals.

13.3 Operational runbooks

- Provider outage: stop retry storms, use fallback only if configured, preserve resumable task state.
- Queue backlog: inspect worker health, rate limits and task age before scaling.
- Vector database outage: fail gracefully, mark retrieval unavailable and avoid fabricating answers.
- Bad model/prompt release: roll back version and rerun evaluation suite.
- Corrupt document: quarantine the ingestion record and preserve the original file for diagnosis.

- Data restore: test PostgreSQL, Qdrant and object-store recovery together.

14. Key design decisions and trade-offs

Decision	Recommendation	Trade-off / reason
Agent framework	LangGraph	Good control over explicit state, routing, parallel nodes and checkpointing; adds graph design complexity.
Vector DB	Qdrant	Dedicated vector search and metadata filtering; requires service operations.
Relational DB	PostgreSQL	Strong transactions and relational integrity for research lifecycle data.
Embeddings	Start with a compact Sentence Transformer	Easy local baseline; evaluate domain-specific alternatives before production scale.
Reranker	CrossEncoder	Can improve precision; adds latency and compute cost.
Search provider	Adapter interface	Avoid provider lock-in; each provider has distinct coverage, cost and terms.
Agent count	Start small; expand from evidence	Reduces cost, complexity and coordination failure.
Execution	Async job + workers	Supports long research runs and durable progress; needs queue operations.
Report format	Structured JSON + Markdown; PDF as renderer	Keeps content separate from presentation.

14.1 Common failure modes to avoid

- Too many agents: parallelism creates overhead, duplicated searches and inconsistent outputs.
- Unbounded loops: critic and verifier should have explicit retry and cost budgets.
- Unstructured state: large prose blobs make merging, testing and recovery difficult.
- Citation afterthought: source IDs and locators must be captured during retrieval.
- False confidence: model self-reported confidence is not a calibrated probability.
- Retrieval without evaluation: embedding similarity alone does not prove relevance.
- Database coupling: agents should not directly manage SQL sessions or vector collections.
- Prompt-injection exposure: source content must never be trusted as system instructions.
- Silent partial failure: missing sources or failed agents must appear in run status and report limitations.

15. Definition of done: production readiness

Area	Acceptance condition
Architecture	Clear boundaries between domain, application, agents, orchestration and infrastructure.
Data	Research runs, tasks, sources, evidence and reports are durable and tenant-scoped.
Retrieval	Hybrid retrieval is evaluated against a reviewed dataset; source locators are preserved.
Agents	Typed contracts, bounded tools, explicit failure states and isolated tests.
Orchestration	Parallel execution, checkpoint recovery, retries, cancellation and budgets work.
Evidence	Material claims are traceable; unsupported and conflicting claims are handled explicitly.
API	Authentication, authorization, rate limits and asynchronous job lifecycle are implemented.
Security	Secrets, uploads, prompt injection, SSRF and cross-tenant access have controls and tests.
Operations	Logs, metrics, traces, alerts, backups, restore drills and runbooks exist.
Quality	Regression evaluation covers retrieval, evidence, citations, coverage, latency and cost.

16. Recommended first vertical slice

Before building the full platform, prove the core research loop with a small but complete implementation.

Vertical-slice scope

- One research request submitted through FastAPI.
- One planner that produces three bounded research tasks.
- One internal PDF collection indexed in Qdrant.
- One external search adapter.
- One document researcher and one web researcher.
- One evidence verifier.
- One synthesizer that produces a cited Markdown report.
- Persist research run, tasks, sources, evidence and report in PostgreSQL.

Vertical-slice success criteria

Check	Pass condition
Traceability	Every factual claim in the report links to a stored evidence record.
Retrieval	The system retrieves relevant passages for a small reviewed set of questions.
Failure handling	A failed external search does not crash the entire run without a recorded reason.
Recovery	A worker restart does not silently lose the research job.
Honesty	Insufficient evidence is reported as a gap, not filled with invented facts.
Reproducibility	The run records model, prompt, embedding and source metadata.

16.1 Immediate build order

Order	Build	Do not move on until...
1	Phase 1 bootstrap	API runs, settings and logs are verified.
2	Phase 2 persistence	Research and task records survive restart.
3	Phase 3 ingestion	A PDF can be extracted with page-level provenance.
4	Phase 4 indexing	Chunks are indexed and filtered correctly.
5	Phase 5 retrieval	Hybrid retrieval beats or complements the baseline on test queries.
6	Phase 6 LLM abstraction	Structured model outputs validate and failures are handled.
7	Phase 7 agents	Each agent returns evidence-linked structured results.
8	Phase 8 graph	A complete run can execute, checkpoint and resume.
9–13	Quality, API and operations	Release gates are satisfied before production exposure.

17. Reference documentation to consult during implementation

Use official documentation for version-specific APIs and operational details. Pin tested versions in the project lockfile rather than assuming that example code remains compatible indefinitely.

Resource	Documentation	Use
FastAPI	https://fastapi.tiangolo.com/	API framework and dependency injection.
Pydantic	https://docs.pydantic.dev/	Validation and settings.
SQLAlchemy	https://docs.sqlalchemy.org/	Async database access and ORM.
Alembic	https://alembic.sqlalchemy.org/	Database migrations.
LangGraph	https://langchain-ai.github.io/langgraph/	Stateful graph orchestration and checkpointing.
Qdrant	https://qdrant.tech/documentation/	Vector collections, filtering and retrieval.
Sentence Transformers	https://sbert.net/	Embedding and CrossEncoder models.
Redis	https://redis.io/docs/	Caching and queue-related infrastructure.
OpenTelemetry	https://opentelemetry.io/docs/	Traces, metrics and observability.

Final engineering recommendation

Build ATLAS as an evidence-first research workflow, not as a collection of autonomous chatbots. The durable value comes from reliable ingestion, retrieval quality, source provenance, explicit verification, bounded orchestration and reproducible reports. Start with the vertical slice, measure its quality, and add agent specialization only when it solves a demonstrated problem.

End of blueprint